Zajel (زاجل) — project briefing

For a second reviewer (Gemini). Written 2026-11-01 from the repository at commit `af7b01c` plus the fixes described in section 6 (they are in the next commit, the one after `af7b01c`). Everything marked **unverified** has not been run on a phone.

> **What is being asked.** One bug is still open after three rounds of phone testing: on the **receiving** phone the new "receiving" screen stays at **0%** and after a few seconds the transfer screen goes away on **both** phones. Section 5 holds everything known about it, section 6 the fix just written, and section 8 the specific questions. Please read the code listed in section 4 and tell us what we are missing.

1. The app in one paragraph

Zajel is an offline, ad-free file-transfer app for Android (iOS and desktop come later), Arabic-first (RTL) with English. It is wire-compatible with the **LocalSend v2.1 protocol** (`https://github.com/localsend/protocol`) and adds optional "Zajel extensions" (parallel chunked upload, tar bundles for many small files, a native Kotlin data path) that are negotiated by HTTP headers, so stock LocalSend apps still work. Flutter 3.38 / Dart 3.10, Riverpod 3, go_router, freezed, Pigeon for all native calls, minSdk 26. About 9,700 lines of Dart, 2,400 of Kotlin, 4,800 of tests (325 tests, all passing; `flutter analyze` clean).

Hard rules (from CLAUDE.md — do not suggest breaking them)

- All transfers use TLS (self-signed certificate per device, identity = SHA-256 fingerprint of the DER certificate). The only plain-HTTP thing is the optional browser share page, which warns the user.
- No ads, no analytics, no network calls to any server. Libraries that bundle telemetry (e.g. Google ML Kit) are forbidden; an automated test scans manifest, pubspec, Dart code and every plugin's Gradle files.
- Native code is reached only through **Pigeon** (no raw MethodChannel strings). On Android the file bytes never pass through Dart on the fast path: Kotlin writes with `FileChannel`; never load a whole file into memory.
- Received file names are sanitised (no `..`, absolute paths, control characters). No exported FileProvider.
- Colors only through ThemeData tokens; all user-visible strings through ARB files (`lib/l10n/app_ar.arb` is the template).

2. Architecture

Area	Files	Role
Discovery	`lib/core/discovery/multicast_discovery.dart`, `device_probe.dart`	UDP multicast `224.0.0.167:53317` announcements; `probeDevice` = fingerprint-pinned HTTPS `GET /info` to check a listed device is alive.
Receive server	`lib/core/transfer/receive_server.dart`	Dart `HttpServer` (TLS): `/info` `/register` `/prepare-upload` `/upload` `/cancel` + Zajel routes (progress, benchmark, tar). Owns the session state machine and emits `ReceiveEvent`s (`SessionStarted`, `FileProgress`, `FileReceived`, `SessionEnded`).
Send client	`lib/core/transfer/send_client.dart`	prepare-upload, then upload: plain stream for small files, tar for many small files, **8 MiB chunks × 4 parallel connections** for big files against a Zajel receiver.
Native data path	`android/.../NativeDataServer.kt`, `lib/core/platform/native_data_bridge.dart`	Kotlin TLS server on a random port (advertised in header `x-zajel-data-port`). Writes chunk bodies straight to the part file; after each chunk asks Dart (`onNativeChunk`) which answers with the HTTP status. **New:**

		`onNativeProgress` every 200 ms.
Service facade	`lib/core/transfer/localsend_service.dart`	Starts server + discovery, accept/decline queue (`PendingIncoming`, 60 s timeout), foreground-service begin/end, owns the `ReceiveTracker`.
Receive UI model	`lib/core/transfer/receive_tracker.dart`	Turns `ReceiveEvent`s into a `ReceiveSnapshot` stream; publishes each finished file to a public folder via Kotlin `publishFile` (MediaStore).
Send UI model	`lib/core/transfer/transfer_providers.dart` (`SendJobNotifier`)	State of the current send, retries (5×, 2 s) on transient errors, speed calculation.
Screens	`lib/features/receive/{incoming_transfer_host,receive_screen}.dart`, `lib/features/send/transfer_progress_screen.dart`	Accept dialog → receiving screen; sender progress screen.
Router-free transports	`lib/core/transport/transport_manager.dart`, `WifiTransport.kt`	Wi-Fi Direct > LocalOnlyHotspot via QR > same LAN, with fallback.

Life of a transfer (Zajel → Zajel, big file)

1. Sender: `POST /prepare-upload` with file list and header `x-zajel-extension`.
2. Receiver (`_prepareUpload`): calls `onIncoming` → UI dialog (accept/decline, 60 s). On accept it creates `_Session`, **emits** **`SessionStarted`**, starts the idle timer (`sessionIdleTimeout` = 2 min, reset by `_touch()`), reserves a `.zajel-part` file per file, registers each with the native server, answers with session id, per-file tokens, `x-zajel-data-port`.
3. `IncomingTransferHost` (after the dialog closes with "accepted") calls `GoRouter.go('/receive')`. Meanwhile the sender has navigated to its own progress screen.
4. Sender uploads 8 MiB chunks over 4 TLS connections to the data port. Kotlin writes each chunk to disk, then `onNativeChunk(sessionId, fileId, offset, length)` → Dart `_commitChunk` records the range in a `RangeSet`, emits `FileProgress`; when the set is complete it renames the part file and emits `FileReceived`.
5. When every file is done: `SessionEnded(cancelled:false)`. `ReceiveTracker` meanwhile calls `publishFile` per finished file (MediaStore copy, then deletes the private copy).

3. Status after the last phone test

Item	Status
LAN transfer via device icon	Works, but ~1 MB/s on the user's router. Via the QR hotspot (direct phone-to-phone) ~30 MB/s. Our reading: router/AP per-client limit (user's network is 2.2.x.x), not the code. **Unverified.** The app now shows Wi-Fi band/link speed and a hint when < 3 MB/s.
Files visible in gallery / file manager	Fixed — user confirmed.
QR pairing, Wi-Fi Direct, speed test, stale devices, selection reset	Changed in round 2. The user then reported "no problem except the receive screen", without saying which transport was used for each check — so treat Wi-Fi Direct as **unconfirmed**.
Receiving screen	**Open bug** — see section 5.

4. Code to read for the open bug

1. `lib/core/transfer/receive_server.dart` — `_prepareUpload` (≈ line 276), `_onNativeChunk` / `_onNativeProgress`, `_commitChunk`, `_finishChunked`, `_endSessionIfFinished`, `_touch` (idle timer), `_abortSession`.
2. `android/app/src/main/kotlin/com/zajel/zajel/NativeDataServer.kt` — `serveOne` read loop and `askDart` (60 s timeout waiting for Dart's answer).
3. `lib/core/transfer/receive_tracker.dart` and `lib/features/receive/receive_screen.dart`; provider `receiveSnapshotProvider` in `transfer_providers.dart`.
4. `lib/features/receive/incoming_transfer_host.dart` (navigation after accept).
5. `lib/core/transfer/send_client.dart` `_uploadChunked` / `_sendChunk` and `SendJobNotifier.send`; `lib/features/send/transfer_progress_screen.dart`.
6. `android/.../TransferService.kt` (foreground service, WifiLock, WakeLock) and `MainActivity.kt` (callback wiring, `publishFile`).

5. The open bug — exactly what we know

> **User report (verbatim, Arabic):** «ولا مشكلة الا مشكلة شاشة الاستقبال عند الطرف الاخر و برضو فيها مشكلة لمن تفتح الشاشة ما ببين تقدم بس بضل 0% و بعد كم ثانية بطفي عند الطرفين»
>
> **Translation:** "No problem except the receiving screen on the other device. Also it has a problem: when the screen opens it shows no progress, it just stays at 0%, and after a few seconds it turns off on both devices." (The same user confirmed that files do arrive and appear in the file manager.)

Facts

- Files *do* arrive and are published to the public folder, so the transfer itself completes at least in some runs.

- We do not know the file sizes used, whether the link was the 1 MB/s router path or the 30 MB/s QR path, or what "turns off" looked like (screen leaves to Home? turns into the "done" state? the whole phone screen sleeps? an error card?). **We have no log from that run.** The app has Settings → export log; `adb logcat -s ZajelWifi flutter` would also help.
- Before the round-2 build, the receiver had no receiving screen at all, so this is the first time anyone looked at receiver-side progress.

Hypotheses, most likely first

#	Hypothesis	Evidence / reasoning
H1	**Receiver progress is chunk-granular on the native path.** Kotlin only tells Dart when a whole 8 MiB chunk is on disk. With 4 parallel chunks, a file ≤ 32 MiB shows 0% until all its chunks land at once, then jumps to 100% and the session ends → screens change within seconds.	Read directly from `_commitChunk` / `NativeDataServer.serveOne`; matches "0%… then gone". The sender's own bar moves because it counts bytes handed to its socket. **Fix written (section 6).**
H2	**Idle timer aborts slow transfers.** `_touch()` was only called from Dart-side events, i.e. once per committed chunk. On a slow link (e.g. 100–300 KB/s) one 8 MiB chunk can take more than the 2-minute `sessionIdleTimeout`; `_abortSession(cancelled:true)` then ends the session on the receiver, and the sender's next request gets 403 / fails → both screens end.	Code reading. Would not be "a few seconds" unless the link is very slow, so H1 explains the symptom better; H2 is a real latent bug either way. **Fix written** (progress callbacks now call `_touch()`).
H3	**Event-stream race.** `receiveSnapshotProvider` is `yield service.receive.current; yield* service.receive.stream;` on a *broadcast* stream. Events emitted in the tiny gap between the two are lost, which could leave the screen stale.	Window is microseconds; low probability, but trivially removable by emitting the current value on subscribe. **Fixed** (section 6).
H4	**Android lifecycle.** The receiver app is in the foreground but the foreground service (`FOREGROUND_SERVICE_DATA_SYNC`) could fail to start on Android 14+ (type/permission/notification permission denied), or the sender's Dart isolate could be throttled when its screen turns off.	No evidence; the phones' Android versions were not recorded. `beginTransfer` swallows exceptions from `startTransferService`.
H5	**Sender cancels.** Any exception in `SendJobNotifier.send` shows an error card (not "closing"), but `cancel()` resets state to `SendJob()` and the screen returns Home. A cancel from the sender would show "cancelled" on the receiver.	No evidence.
H6	**Publish step.** `ReceiveTracker._save` awaits Kotlin `publishFile` (copy to MediaStore, delete source). If it throws or is slow, the file stays "saving" and the session end waits (`isOver` needs no file in `saving`).	User says files appear in the file manager, so publishing works at least for them.

6. Changes made after commit af7b01c

- **Live receive progress.** Pigeon `NativeDataCallbacks.onNativeProgress(sessionId, fileId, liveBytes)`. Kotlin keeps an `AtomicLong live` per upload, adds every buffer it writes, reports at most every 200 ms, and subtracts the chunk once it is fully written (just before asking Dart about the chunk). Dart `_onNativeProgress` emits `FileProgress(covered + liveBytes)` and calls `_touch()`.
- `ReceiveTracker` makes the receiving bar monotonic (between "chunk written" and "chunk committed" the two numbers are briefly out of step).
- Test added: with the fake native bridge, progress values strictly between 0 and one chunk size are seen and never decrease. All 325 tests pass; the Kotlin changes compile but have **not run on a phone**.
- `receiveSnapshotProvider` now subscribes to the tracker stream *before* adding the current value (H3).
- One extra log line when a file is received.

7. How to build, run, test

```
flutter pub get
flutter analyze
flutter test                    # 325 tests; slow/network ones are opt-in: ZAJEL_SLOW=1 /
ZAJEL_NETWORK=1
dart run pigeon --input pigeons/platform_api.dart   # regenerates
lib/core/platform/platform_api.g.dart and PlatformApi.g.kt
flutter build apk --release --target-platform android-arm64
```

Windows dev machine, no Mac, only two physical Android phones. Wi-Fi Direct, hotspots, MediaStore and the Kotlin data server cannot be tested without them; the Dart side of the native path is tested with `test/support/fake_native_data_bridge.dart`. Docs: `docs/SPEC.md` (acceptance criteria), `docs/ZAJEL_EXTENSIONS.md` (our protocol additions), `docs/TRANSPORTS.md`, `docs/TESTING.md` (hand-test checklist), `docs/BENCHMARKS.md`.

8. What we would like from you

1. Read sections 2 and 4 and the listed code. **Find a cause of "0% then both screens end" that we have not listed**, or confirm H1/H2 and say what else would be needed to make the receive screen trustworthy.
2. Review `NativeDataServer.serveOne` and `ReceiveServer._onNativeChunk/_onNativeProgress` for races or leaks (live-byte accounting when a connection drops mid-chunk, a retried chunk, two connections writing different chunks of one file).
3. Review the session lifecycle (`_touch`, `_abortSession`, `_endSessionIfFinished`, `PendingIncoming` 60 s timeout, `IncomingTransferHost` navigation) for a path that ends the session or the screen early.
4. Suggest the smallest instrumentation (log lines or on-screen debug text) that would make the next phone test conclusive, given that we can only ask the user to send an exported log.
5. Any Android-specific pitfalls you know for: foreground service type `dataSync` on Android 14/15, `MediaStore` publishing from a background thread, `WifiNetworkSpecifier` + `bindProcessToNetwork`,

Wi-Fi Direct DNS-SD discovery (we call `discoverPeers` plus repeated `discoverServices` every 4 s).

6. Please do not propose anything that adds a server, telemetry, cleartext transfer, or a Google/Firebase dependency.

Repository layout and the full history are in git (11 commits). The uploaded repository is private; the user will hand you the zip.